Departamento de Ingeniería Informática
Área de Lenguajes y Sistemas Informáticos
GRADO EN DISEÑO DE VIDEOJUEGOS.

Ficha	Práctica 02
Versión	1.2
Fecha	16-10-2026

DESARROLLO DE EXPERIENCIAS DE REALIDAD VIRTUAL

Práctica 2: Dispositivos de Entrada Innovadores – Abstracción de Hardware y Conectividad en Unreal Engine 5

INTRODUCCIÓN.

Los dispositivos de entrada son la primera capa de cualquier experiencia interactiva: un buen diseño de juego puede arruinarse si el control responde tarde o si un botón no hace lo que el jugador espera. En las experiencias inmersivas y en las instalaciones tipo *arcade*, además, es habitual trabajar con hardware que no es un mando de consola estándar: paneles de botones, joysticks de recreativa, sensores conectados por puerto serie, etc.

En esta práctica construirás una **mesa de pinball jugable** dentro de un nivel de Unreal Engine 5. El jugador se acercará a la máquina, tomará el control de la mesa y podrá accionar los *flippers* y el lanzador tanto con el teclado como con un **controlador USB arcade genérico**. El objetivo no es solo que el pinball funcione, sino que la lógica del juego sea **independiente del dispositivo físico**: la mesa reacciona a *acciones* (lanzar, flipper izquierdo, flipper derecho), nunca a teclas o botones concretos.

OBJETIVOS TÉCNICOS.

El alumno aprenderá a desacoplar la capa de hardware físico de la lógica de juego utilizando el **Enhanced Input System (EIS)** de Unreal Engine 5. Comprenderá cómo mapear señales analógicas y digitales crudas provenientes de diferentes protocolos de comunicación (HID estándar, emulación XInput y comunicación serie por puerto COM).

Al finalizar la práctica, el alumno será capaz de:

- Crear **Input Actions** y agruparlas en un **Input Mapping Context** propio para cada modo de juego.
- Asignar a una misma acción entradas de dispositivos distintos (teclado y controlador HID genérico).
- Cambiar dinámicamente el contexto de entrada al **poseer** (*possess*) un nuevo *pawn*.
- Traducir las acciones en respuestas físicas (rotaciones animadas, impulsos, rebotes) mediante *Blueprints*.

MÓDULOS DE HARDWARE A IMPLEMENTAR.

1. Controlador USB Arcade (Capa HID Estándar)

Implementación: configuración directa mediante el mapeo de botones genéricos a través del protocolo nativo de Windows e inyección en el *Enhanced Input System*.

Foco en el diseño: crear un esquema de control clásico de baja latencia. El alumno debe configurar los archivos de mapeo de entrada (*Input Mapping Contexts*) para resolver el problema de la asignación de botones en dispositivos que no siguen el estándar XInput (Xbox).

OPERATORIA.

Parte del proyecto base de la práctica (plantilla *First Person*) que ya contiene la carpeta **Content/GameArcade** con el modelo de la máquina de pinball. El trabajo se organiza en seis fases; cada una se apoya en la anterior, por lo que conviene probar el juego (*Play In Editor*) al terminar cada fase antes de continuar.

> **Antes de empezar:** conecta el controlador USB arcade y comprueba en Windows (*Configurar dispositivos de juego USB*, comando **joy.cpl**) qué número de botón envía cada pulsador. Unreal los reconocerá como **GenericUSBController Button N**. Comprueba también que ningún botón aparezca pulsado sin tocarlo.

Qué incluye ya el proyecto base

El modelo de la mesa procede de Visual Pinball y algunas piezas no se exportaron bien. Para que puedas centrarte en la entrada y en la lógica de juego, el proyecto base ya trae resuelto lo siguiente (no tienes que tocarlo):

- **Colisiones de la mesa.** Las mallas usan su geometría real como colisión (*Use Complex Collision As Simple*); los rollovers de alambre, las bombillas y los cables de las puertas no colisionan.
- **Piezas que faltaban**, sustituidas por cajas invisibles: la rampa y el tramo elevado del carril de salida (*lanzador1*, *carrilelevado*), una puerta de un solo sentido (*unsentido*), varias guías, las gomas de la isla central y un cristal (*tapacristal*) que impide que la bola se salga de la mesa.
- **Kickers** en los cuatro agujeros de la cubierta amarilla y en los dos laterales: atrapan la bola y la devuelven al juego. Funcionan con cualquier objeto que simule física, así que valdrán para tu bola.
- **Pivotes de los flippers** (*pivoteizq* y *pivotedch*), ya orientados según la inclinación de la mesa. Cada flipper cuelga de su pivote, de modo que para moverlo basta con girar su eje Yaw.
- **Bumpers preparados:** el anillo y la base de cada uno tienen activado *Simulation Generates Hit Events* y la etiqueta (*Component Tags*) **bouncer**. Las dianas y las gomas de los pasillos también tienen los eventos de golpe activados y las etiquetas **diana** y **goma**, listas por si quieres que puntúen.
- Un componente **spawn** en el carril de salida, que usarás para reiniciar la bola, y la zona del lanzador (**lanzador**) con la flecha (**Arrow**) que marca la dirección del lanzamiento, ya inclinada como la mesa.
- **Física con substepping** (*Project Settings › Engine › Physics › Substepping*): *Max Substep Delta Time* 0,004167 s (240 Hz) y *Max Substeps* 8. La bola es pequeña y rápida, y los flippers la golpean en unos milisegundos: sin subpasos puede atravesar piezas finas o recibir golpes imprecisos. Este ajuste se guarda en *Config/DefaultEngine.ini*, así que forma parte de lo que debes subir a GitHub.
- La cámara del pinball (**Camera**), que pasa a ser la vista del jugador al poseer la mesa, y el plugin **Raw Input** activado, necesario para que Unreal reconozca el controlador arcade como *GenericUSBController*.

1. Tomar el control del pinball (Possess)

El jugador comienza controlando al personaje en primera persona. Cuando se acerque a la máquina, el control debe pasar a la mesa de pinball y el sistema de entrada debe cambiar al esquema de controles del pinball.

- En el *Blueprint* del pinball (*Pawn*), añade una **Box Collision** delante de la máquina (en el ejemplo, *triggerposses*).
- En el evento **On Component Begin Overlap** de ese trigger, comprueba con un **Branch** que el actor que entra (*Other Actor*) es el *pawn* del jugador (**Get Player Pawn**).
- Si es así, llama a **Possess** usando el **Player Controller** como *Target* y el propio pinball (**Self**) como *In Pawn*.
- Por último, haz un **Cast** al *BP_FirstPersonPlayerController* y utiliza **Change Mapping Context** para activar el contexto **IMC_pinball** que configurarás en la fase 2.
- Dentro del *BP_FirstPersonPlayerController*, ese evento personalizado (**ChangeMappingContext**) obtiene el *Enhanced Input Local Player Subsystem*, llama a **Clear All Mappings** y después a **Add Mapping Context** con **IMC_pinball** y *Priority* 1.

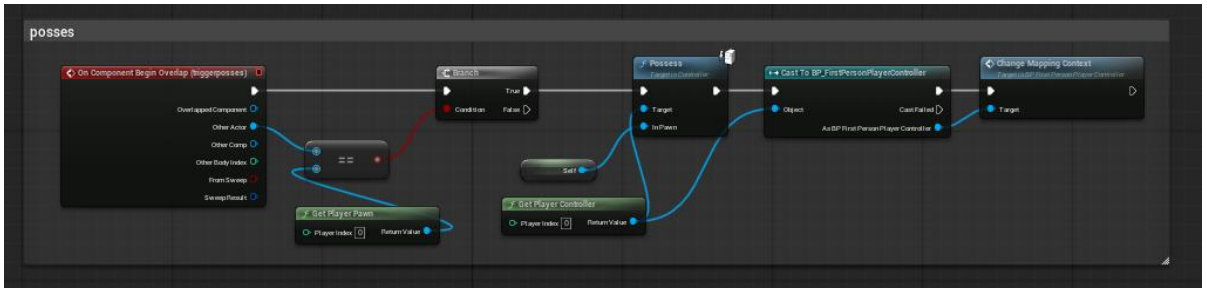


Figura 1. Cambio de pawn y de contexto de entrada al acercarse a la máquina.

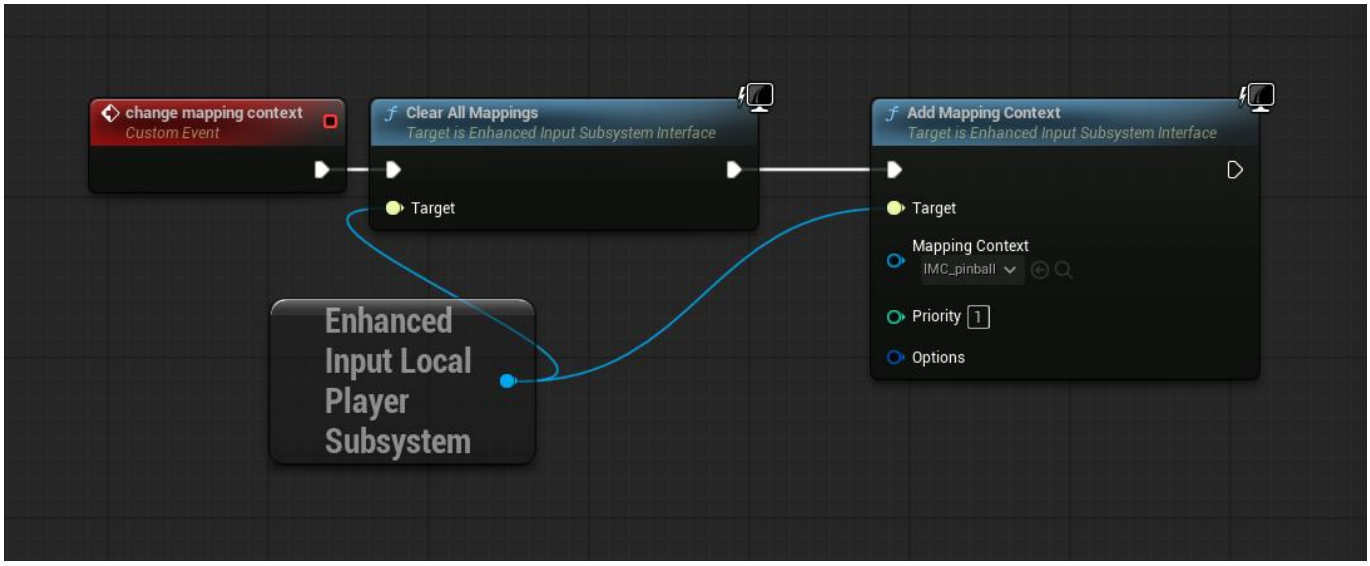


Figura 2. Evento ChangeMappingContext.

> **Atención:** el contexto del personaje (*IMC_Default*) también usa las flechas para moverse (*IA_Move*). Si no lo quitas antes de añadir *IMC_pinball*, ambas acciones compiten por las mismas teclas y los flippers pueden no responder.

2. Crear las acciones de entrada (Enhanced Input)

Esta es la fase central de la práctica. Vas a definir **qué puede hacer** el jugador (acciones) por separado de **con qué lo hace** (mapeo de teclas y botones).

- Crea en la carpeta **Input** cuatro *Input Actions*: **lanzador**, **blipper_izqd**, **blipper_dcha** y **reiniciar** (devuelve la bola al carril de salida).
- Configura cada acción con **Value Type: Digital (bool)** y añade un *trigger* **Hold And Release** (*Hold Time Threshold* 0,01; *Actuation Threshold* 0,5), de modo que la acción notifique tanto la pulsación como la liberación del botón.

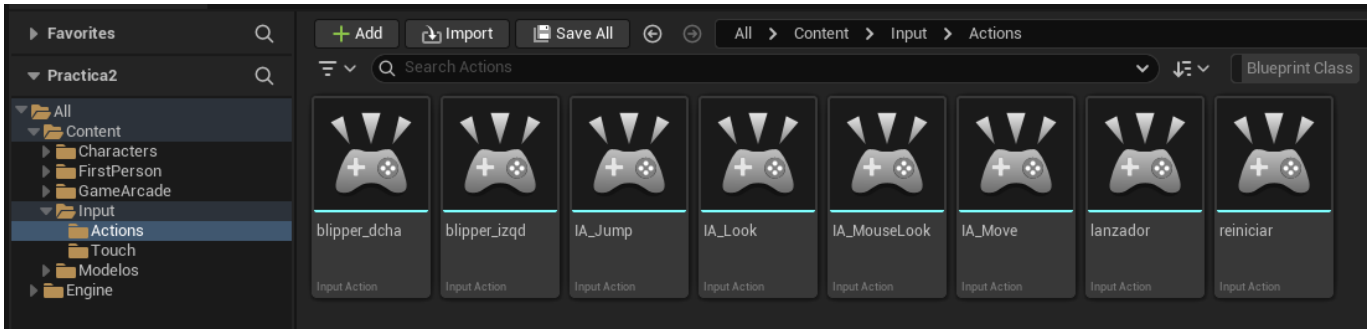


Figura 3. Input Actions del pinball.

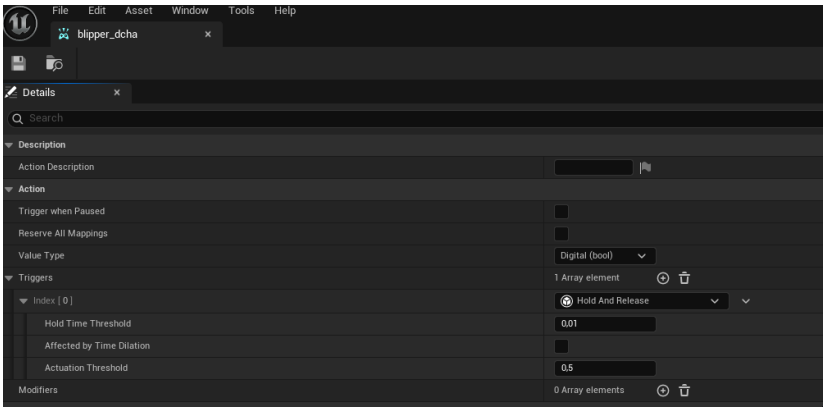


Figura 4. Configuración de la Input Action del flipper derecho.

- Edita el *Input Mapping Context* **IMC_pinball** y asigna a cada acción una tecla (*Left*, *Right*, *Up*) **y** el botón correspondiente del controlador arcade (**GenericUSBController Button N**). En la figura solo aparece mapeado un botón del controlador: debes completar el mapeo de todas las acciones.
- Asigna **reiniciar** a la tecla **R** y a un botón libre del controlador arcade. Para comprobar que la lógica no depende del dispositivo, añade también un mando XInput: *Gamepad Left Shoulder*, *Gamepad Right Shoulder*, *Gamepad Face Button Bottom* y *Gamepad Special Right*.

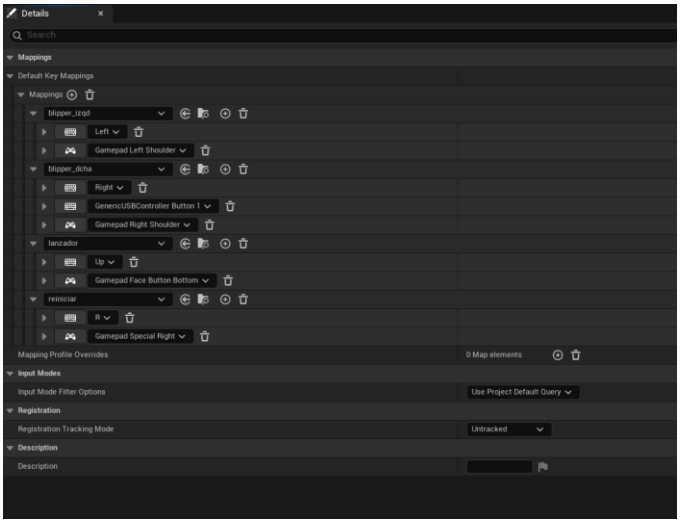


Figura 5. Input Mapping Context con entradas de teclado y de controlador USB genérico.

- En el *Event Graph* del pinball, usa los eventos **EnhancedInputAction** de cada flipper: la salida *Started* reproduce (**Play**) un **Timeline** y la salida *Completed* lo invierte (**Reverse**). Conecta también la salida *Canceled* a **Reverse**: si el botón se suelta antes de tiempo, el flipper vuelve igualmente a su sitio.
- La pista del *Timeline* alimenta **Set Relative Rotation** (eje *Yaw*) del componente del flipper, de forma que el flipper sube mientras el botón está pulsado y vuelve a su posición al soltarlo.

- Gira el flipper, no su pivote. En reposo su rotación relativa es 0, así que la pista de cada *Timeline* (0,1 s de duración) debe ir de **0 a −50** en el flipper izquierdo y de **0 a 50** en el derecho; conéctala al Yaw de *New Rotation*.

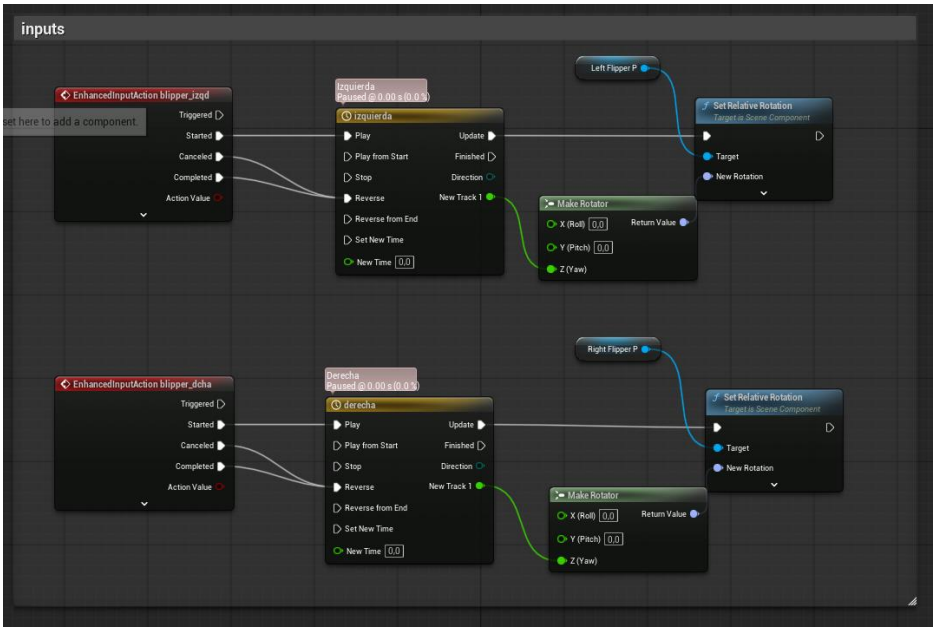


Figura 6. Accionamiento de los flippers mediante Timelines.

> **Clave de diseño:** la lógica de los flippers no sabe nada de teclas ni de botones. Si mañana se sustituye el controlador arcade por otro dispositivo, solo habrá que modificar **IMC_pinball**, sin tocar ningún *Blueprint*.

3. Crear la bola y su comportamiento físico

Un pinball se juega con la física: la sensación de juego depende de cuánto pesa la bola, cuánto rebota y cuánto rozamiento tiene contra la mesa.

- Crea en **Content/Modelos** un *Blueprint* **bola** con un componente esfera.
- Escala la esfera a **0,025** (una bola de 2,5 cm, como la real): los pasillos, los agujeros y los kickers de la mesa están hechos para ese tamaño. Coloca la bola en el nivel, en el carril de salida.
- En el *Blueprint* del pinball, crea una variable **bolachild** de tipo *bola* (*Object Reference*) y márcala como *Instance Editable*. Selecciona el pinball en el nivel y, en el panel *Details*, asígnale la bola que acabas de colocar: la usarás en las fases 4 y 5 para saber si lo que toca la mesa es la bola.

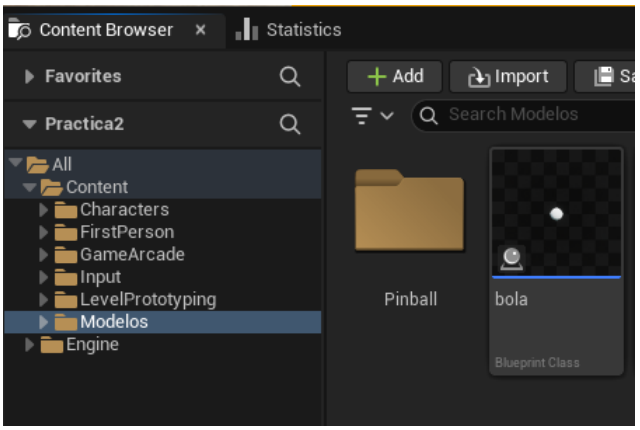


Figura 7. Blueprint de la bola en el Content Browser.

- Activa **Simulate Physics** y ajusta **Mass** (0,5 kg), **Linear Damping** (0,01) y **Angular Damping** (0,0).

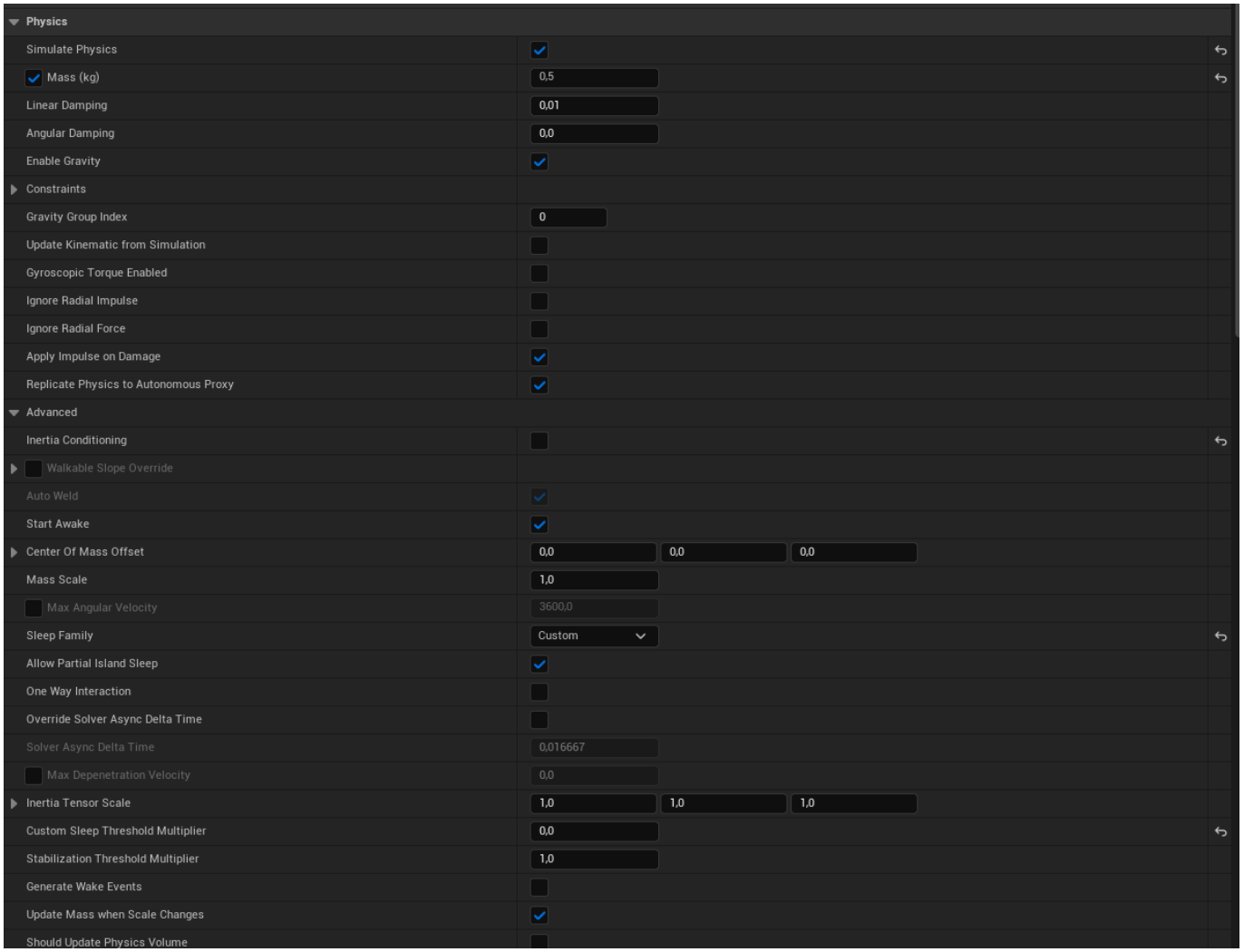


Figura 8. Parámetros físicos de la bola.

- En **Collision**, activa *Simulation Generates Hit Events* y *Generate Overlap Events*, usa el perfil **PhysicsActor** y asigna un material físico propio en *Phys Material Override*.
- En las opciones avanzadas de física de la esfera, activa **CCD**, desactiva **Inertia Conditioning** y pon **Sleep Family** en *Custom* con **Custom Sleep Threshold Multiplier** a 0.

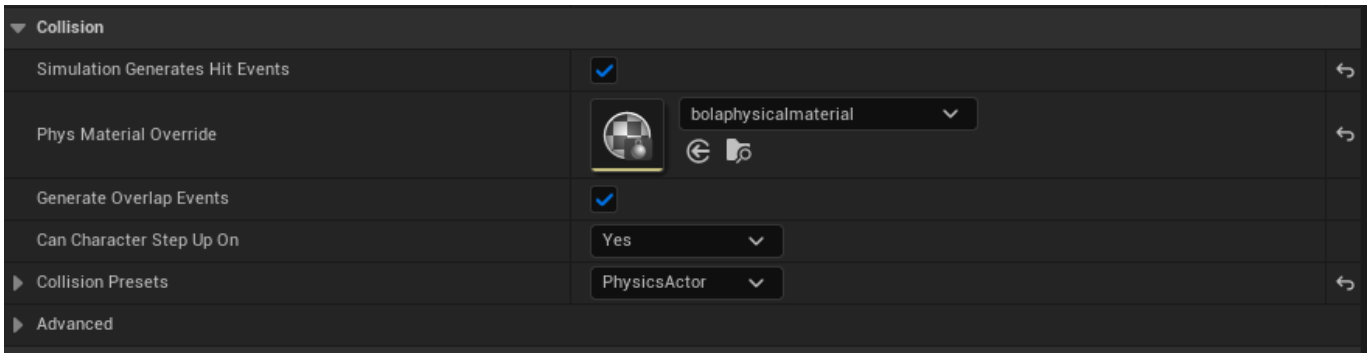


Figura 9. Configuración de colisiones de la bola.

- Crea el **Physical Material** *bolaphysicalmaterial* con rozamiento bajo (*Friction* 0,05; *Static Friction* 0,0) y **Restitution** 0,75 para que la bola rebote de forma viva.

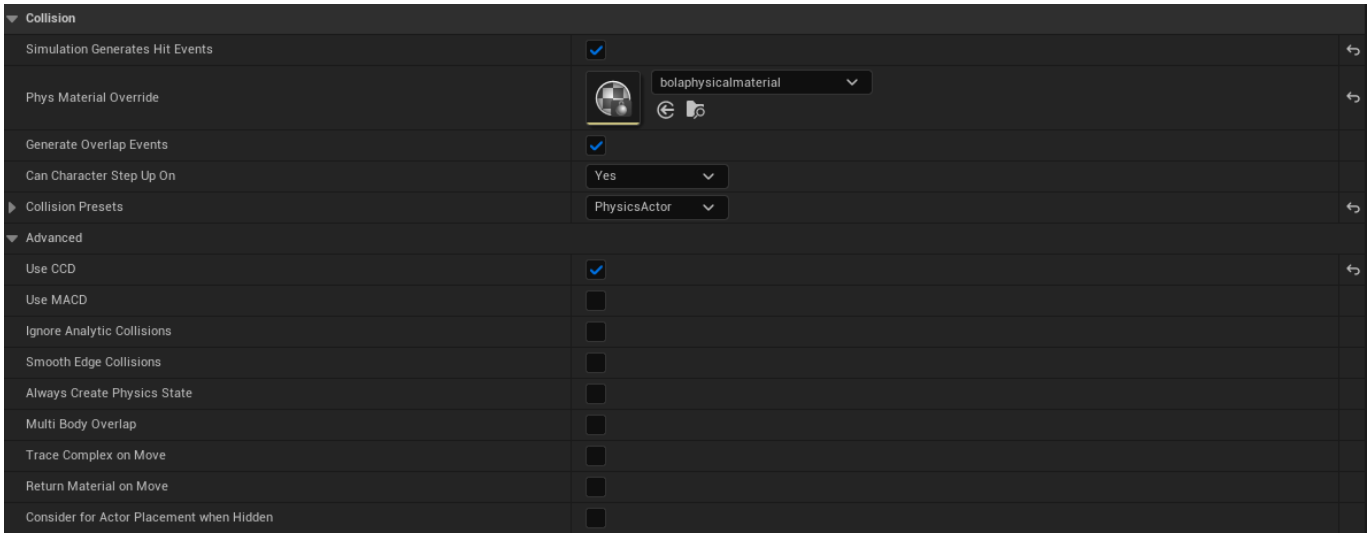


Figura 10. Material físico de la bola.

Estos valores son un punto de partida: ajústalos hasta que el comportamiento de la bola resulte creíble.

> **¿Por qué?** La bola mide 2,5 cm y se mueve deprisa. *CCD* evita que atraviese piezas finas; *Inertia Conditioning* aumenta artificialmente la inercia de los objetos pequeños y la hace rodar a cámara lenta; y si se le permite «dormirse», el motor la congela cuando se para un instante en mitad de la pendiente.

4. Crear los bouncers (rebotadores)

Los *bouncers* son los elementos que, al ser golpeados, expulsan la bola con fuerza. Deben empujarla alejándola de su centro y siempre en el plano de la mesa.

- El **Event Hit** del pinball se lanza con cualquier componente que tenga activados los eventos de golpe. Empieza con un **Branch** que compruebe que el actor golpeado (*Other*) es la bola y que el componente (*My Comp*) tiene la etiqueta **bouncer** (*Component Has Tag*).
- Calcula la dirección restando a la posición de la bola (**Get World Location**) el centro del *bouncer*, que obtendrás con **Get Component Bounds** (salida *Origin*): las piezas importadas tienen el pivote en el origen de la mesa, así que su *Get World Location* no sirve.
- Proyecta ese vector sobre el plano de la mesa (**Project Vector on to Plane**, usando el **Up Vector** del *Playfield*) para que la bola no salga despedida hacia arriba, y **normalízalo**.
- Multiplica la dirección por la fuerza deseada (en el ejemplo, 100) y aplícala con **Add Impulse** sobre la bola, con *Vel Change* activado.
- Protege el evento con un **Do Once** que se reinicia tras un **Delay** de 0,2 s, para evitar que un mismo choque aplique varios impulsos seguidos.

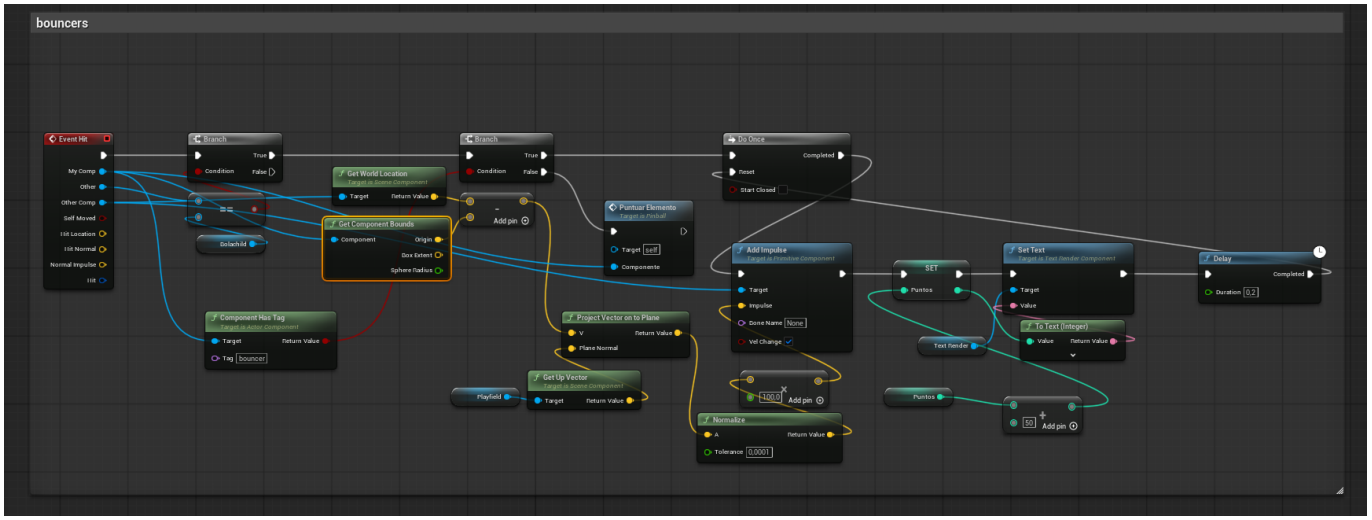


Figura 11. Lógica de rebote de los bouncers.

5. Crear el lanzador (launcher)

Por último, el jugador necesita poner la bola en juego. El lanzador solo debe actuar cuando la bola está en su posición de salida.

- Usa el evento **EnhancedInputAction lanzador**, salida *Completed* (al soltar el botón).
- Comprueba con **Is Overlapping Actor** que la bola está dentro de la zona del lanzador; si no lo está, no ocurre nada.
- Aplica un **Add Impulse** a la esfera de la bola en la dirección de un componente **Arrow** colocado en el carril de salida (**Get Forward Vector** multiplicado por la fuerza de lanzamiento, en el ejemplo -500).

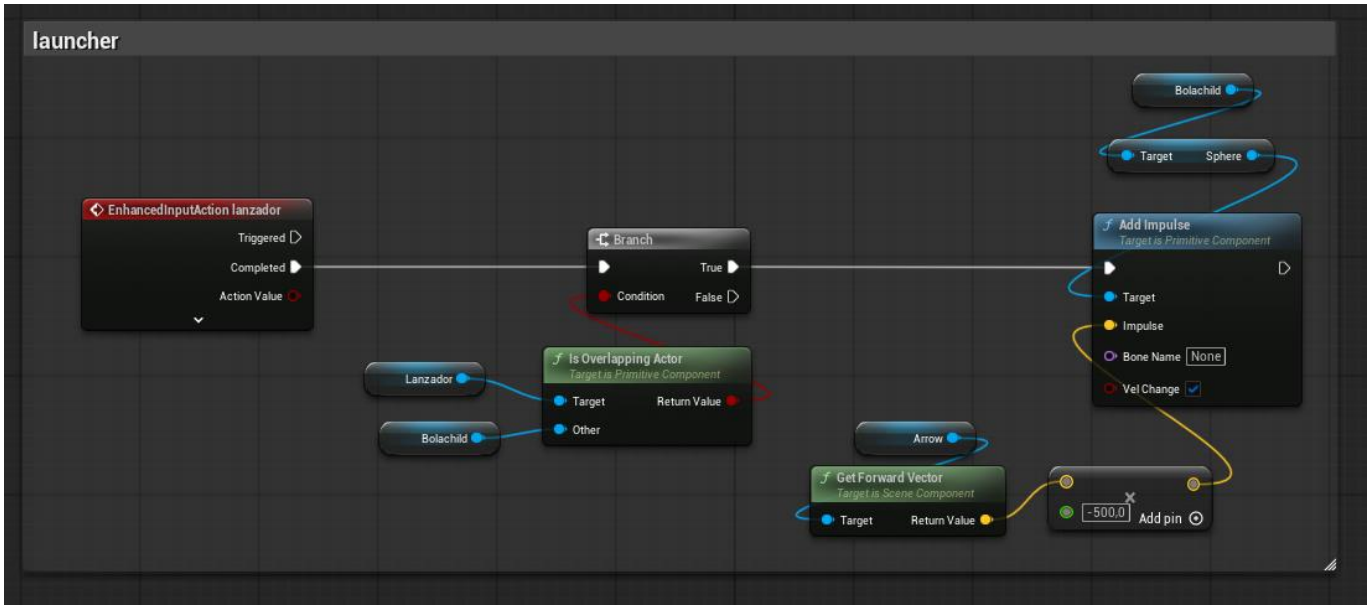


Figura 12. Lógica del lanzador de la bola.

Reinicio de la bola. Para volver a poner la bola en el carril sin salir del juego:

- Usa el evento **EnhancedInputAction reiniciar**, salida *Started*.
- Desactiva **Simulate Physics** en la esfera de la bola, llévala a la posición del componente **spawn** (**Set Actor Location**) y, tras un **Delay** de 0,2 s, vuelve a activar la simulación: así pierde la velocidad que llevaba.

Con esta fase completada, el pinball debe ser plenamente jugable con teclado y con el controlador arcade.

6. Sistema de puntuación

Un pinball sin marcador pierde buena parte de su gracia: la puntuación es lo que da objetivo a cada partida y recompensa al jugador por mantener la bola en juego. En esta fase añadirás un contador de puntos visible en la propia máquina.

- En el *Blueprint* del pinball, crea una variable **Puntos** de tipo *Integer* inicializada a 0.
- Añade un componente **Text Render** en la máquina (por ejemplo, en el panel superior o *backglass*) donde se mostrará el marcador.
- En la lógica de los *bouncers*, justo después del **Add Impulse**, suma los puntos del impacto a la variable (en el ejemplo, **Puntos + 50**) y guarda el resultado con **SET**.
- Actualiza el marcador con **Set Text** sobre el *Text Render*, convirtiendo el valor con **To Text (Integer)**.
- Al estar dentro del **Do Once**, cada impacto suma una sola vez aunque la bola permanezca en contacto con el *bouncer* varios fotogramas.

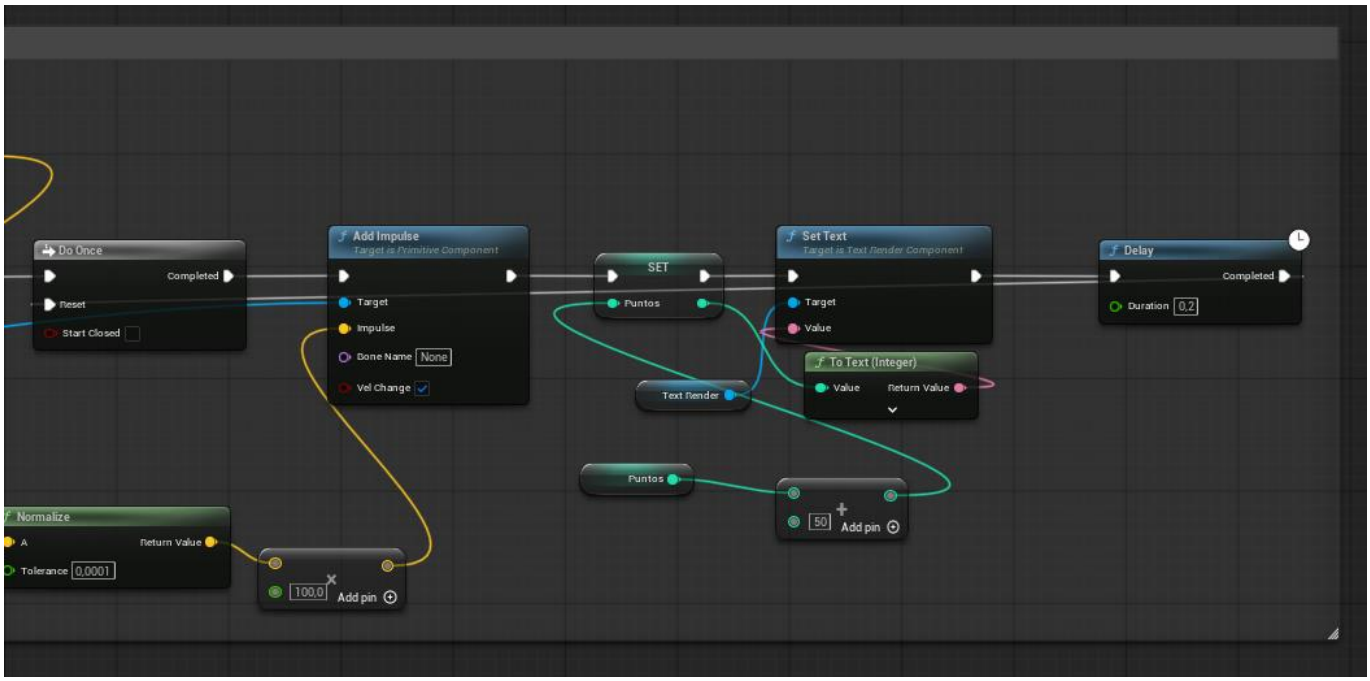


Figura 13. Suma de puntos y actualización del marcador en cada impacto con un bouncer.

A partir de aquí puedes ampliar el sistema: distintos valores según el elemento golpeado, multiplicadores, bonus al activar varias dianas, récord de la partida, etc. El proyecto base también te da un punto de partida: puedes hacer que puntúen las dianas y las gomas (ya tienen las etiquetas diana y goma, así que basta con comprobarlas en el mismo Event Hit), sumar puntos cuando la bola caiga en un agujero de la cubierta o en un kicker lateral, o añadir una red anti-atascos que la empuje si se queda parada demasiado tiempo.

CONDICIONES DE ENTREGA.

El proyecto debe entregarse completo, con todas las configuraciones y objetos importados listos para su ejecución. Debido a su gran tamaño, debe utilizarse **GitHub**. El repositorio debe incluir un fichero **README** con:

- La tabla de controles: cada *Input Action* con su tecla y su botón del controlador arcade (y del mando XInput, si lo usas).
- Los valores físicos finales de la bola y del material físico.
- La lista de mejoras implementadas (efectos, *game feel*, escenario, nuevos elementos…).

Cómo subir el proyecto a GitHub

GitHub no es solo el medio de entrega: es tu herramienta de trabajo durante toda la práctica.

- **Crea el repositorio al empezar**, no al final, y trabaja sobre él todo el rato: haz *commit* y *push* al terminar cada fase (o cada vez que algo funcione), con mensajes que expliquen lo que has hecho. El historial de *commits* forma parte de la entrega y muestra cómo has ido avanzando.

- **Antes del primer commit**, copia en la raíz del proyecto (junto al *.uproject*) el fichero **.gitignore** de ejemplo que tienes en UBUVirtual. Excluye las carpetas que Unreal regenera por su cuenta (*Binaries*, *Intermediate*, *Saved*, *DerivedDataCache*), que pesan mucho y no deben subirse.
- Sube el proyecto completo: el fichero **.uproject**, las carpetas **Config** y **Content** (con los modelos importados) y el **README**.
- GitHub rechaza los ficheros de más de 100 MB: NO deberías tener *asset* que supere ese tamaño.
- Antes de entregar, comprueba que el repositorio está completo: clónalo en otra carpeta y abre el *.uproject*; la mesa debe funcionar sin hacer nada más.

El enlace deberá subirse a **UBUVirtual** en la actividad "Entrega de Práctica 02". La fecha límite será la indicada en UBUVirtual.

La no observación de estas condiciones de entrega, así como la entrega fuera de plazo, son condiciones excluyentes para su evaluación, e invalida la misma de cara a obtener una valoración.

CRITERIOS DE EVALUACIÓN.

La práctica se calificará sobre 10 puntos. Implementar correctamente lo pedido permite alcanzar el aprobado con holgura; para optar a la máxima nota hay que ir más allá y **mejorar la experiencia de juego**.

Bloque	Se valora	Puntos
Implementación de lo pedido	El pinball funciona completo siguiendo las seis fases: toma de control de la máquina, mapeo de entradas con teclado y controlador USB arcade, flippers, física de la bola, bouncers, lanzador y marcador de puntos.	hasta 6
Mejoras y game feel	Todo lo que haga que jugar resulte más satisfactorio: efectos al rebotar (partículas, sonido, luces, animación del bouncer al ser golpeado), vibración o sacudida de cámara, feedback visual al sumar puntos, ajuste de la física, nuevos elementos de juego (rampas, dianas, reinicio de bola…).	hasta 3
Escenario y presentación	Ambientación del nivel y de la máquina: iluminación, materiales, decoración del entorno, organización del proyecto y README completo.	hasta 1
Total		**10**

Las mejoras son abiertas: se valorará la creatividad y el cuidado del resultado final, no la cantidad. Describe en el README qué mejoras has añadido para que puedan tenerse en cuenta.